TASK 5

Numerical Precision

A float32 versus float64 study for a two-dimensional particle simulation

FUNDAMENTALS OF SCIENTIFIC COMPUTING | ATOMIC TOY MODEL PROJECT

Purpose

This experiment compares two floating-point formats while keeping the physical model, initial particle configuration, timestep, integration method, and simulation duration fixed. The comparison measures memory, execution time, total-energy drift, and trajectory difference.

Executive summary

For the 2D model with 12 particles, the tracked working arrays required 2,208 bytes in float32 and 4,272 bytes in float64. This is a 1.935x memory ratio. When the saved position history is included as a separate allocation, the totals are 194,304 and 388,464 bytes, respectively.

The measured median runtime was 0.210111 s for float32 and 0.196522 s for float64. The difference is very small compared with run-to-run variation, so this small-N experiment does not establish a reliable speed winner. Maximum relative energy drift was 0.3029% for float32 and 0.3125% for float64. The final aggregate relative position difference between the trajectories was 49.89%.

Main conclusion

float32 nearly halves the storage required by floating-point arrays, but it did not show a dependable speed advantage for this 12-particle test. The energy histories remain close, while the particle-by-particle trajectories diverge over time. float64 is the safer default when reproducibility and trajectory fidelity are priorities; float32 may be useful when memory pressure becomes important, subject to validation.

Prepared as the Task 5 numerical-precision section of the group project report.

1. Model and simulation setup

The precision comparison uses the two-dimensional quantitative model in the uploaded task3_4.py file. The simulation region is a square [0,L] x [0,L]. Each particle has an x- and y-coordinate, two velocity components, and a mass. Particles interact through the cutoff-based spring force; particles are reflected from the four box walls.

Model consistency note

The uploaded simulator.py is also a 2D animation, but it uses a separate soft-repulsion / hard-core-collision variant (N=16, radius=0.18, rc=0.90, k=28). The quantitative Tasks 3-4 file task3_4.py uses N=12, k=15, r0=rc=1.5. This Task 5 report follows task3_4.py so its energy and integrator model stay aligned with the quantitative Tasks 3-4 work. Do not combine the two force models when reporting results.

1.1 Parameters used

Parameter	Value	Meaning
N	12	Number of particles
Mass	1.0	Mass of each particle
Dimension	2	Position and velocity shape: (12, 2)
L	10.0	Side length of square box
k	15.0	Spring stiffness
r0	1.5	Preferred pair spacing
rc	1.5	Interaction cutoff; equal to r0
dt	0.005	Time step
T	10.0	Simulation duration, matching Task 3 duration
Seed	42	Reproducible initial state
gamma	0.0	No viscous drag; conservative comparison
Integrator	Velocity-Verlet	Same smooth integrator as task3_4.py

1.2 Governing equations

- Pair displacement: d_ij = r_i - r_j, where r_i = (x_i, y_i).
- Pair distance: r_ij = sqrt((x_i - x_j)^2 + (y_i - y_j)^2).
- For neighbors with 0 < r_ij <= rc: F_ij = -k (r_ij - r0) d_ij / r_ij. The net force on particle i is the sum over its neighbors.
- Kinetic energy: K = (1/2) sum_i m_i |v_i|^2.
- Potential energy: U = sum over unique interacting pairs (1/2) k (r_ij - r0)^2. Each interacting pair is counted exactly once.
- Total mechanical energy: E = K + U.

The force and potential use r0 = rc = 1.5, so the pair potential reaches zero at the cutoff. The same force law, wall-reflection rule, and Velocity-Verlet sequence are used for both precision runs.

2. Experimental method

The two runs were designed to differ only in floating-point representation:

- Generate one reproducible initial state in float64 using NumPy default_rng(42).
- Convert copies of the same positions, velocities, and masses to float32 and float64.
- Run both copies with identical N, L, k, r0, rc, dt, duration, wall treatment, and Velocity-Verlet integration.
- Set gamma = 0 so that viscous dissipation does not obscure the precision comparison.
- Store K(t), U(t), E(t), and the particle positions at every step. Use these data to compare energy histories and trajectories.
- Warm up both code paths, then time each precision five times in alternating order. Report the median wall time.

2.1 Quantities measured

Quantity	Definition / method
Floating-point storage	float32 = 4 bytes per value; float64 = 8 bytes per value.
Working-array memory	Sum of nbytes for positions, velocities, masses, forces, pair displacements, pair distances, and the Boolean neighbor mask.
Saved trajectory memory	(number of time frames) x N x 2 x itemsize.
Execution time	Median of five measured wall-clock runs after warm-up.
Maximum energy drift	epsilon_E,max = max_t |E(t) - E(0)| / |E(0)|.
Relative position difference	epsilon_pos(t) = ||x32(t) - x64(t)|| / ||x64(t)||, where all comparison arithmetic is evaluated in float64.

2.2 Memory accounting details

For the pairwise displacement array d with shape (N, N, 2), memory is N^2 x 2 x itemsize. With N=12, d contains 288 floating-point values: 1,152 bytes in float32 and 2,304 bytes in float64. The distance matrix contains 144 floating-point values; the Boolean neighbor mask contains 144 Boolean values and therefore does not double in size when the precision changes.

The working-array total is an explicit NumPy nbytes accounting, not a measurement of total process RAM. It excludes Python object overhead and temporary arrays created during expressions. The stored position history is therefore reported separately.

2.3 Reproducibility

The standalone script task5_precision_2d.py reproduces the experiment and writes CSV files, plots, and environment information into task5_2d_results/. The random seed and all model settings are defined near the top of the script.

3. Results

3.1 Memory usage

Memory category	float32 (bytes)	float64 (bytes)	float64 / float32
Core state arrays	336	672	2.000x
Pairwise arrays	1,872	3,600	1.923x
Tracked working-array total	2,208	4,272	1.935x
Saved position history	192,096	384,192	2.000x
Working arrays + position history	194,304	388,464	1.999x

For the tracked working arrays, float64 uses 1.935 times the memory of float32. When the saved position history is included, float64 uses 1.999 times as much memory. This is close to a factor of two because most stored values are floating-point numbers, while the Boolean neighbor mask has the same size in both runs.

3.2 Energy behaviour

Energy metric	float32	float64
Initial E(0)	77.14255524	77.14255075
Final E(T)	76.91017914	76.90202226
Maximum relative drift	3.028613e-03 (0.3029%)	3.124846e-03 (0.3125%)

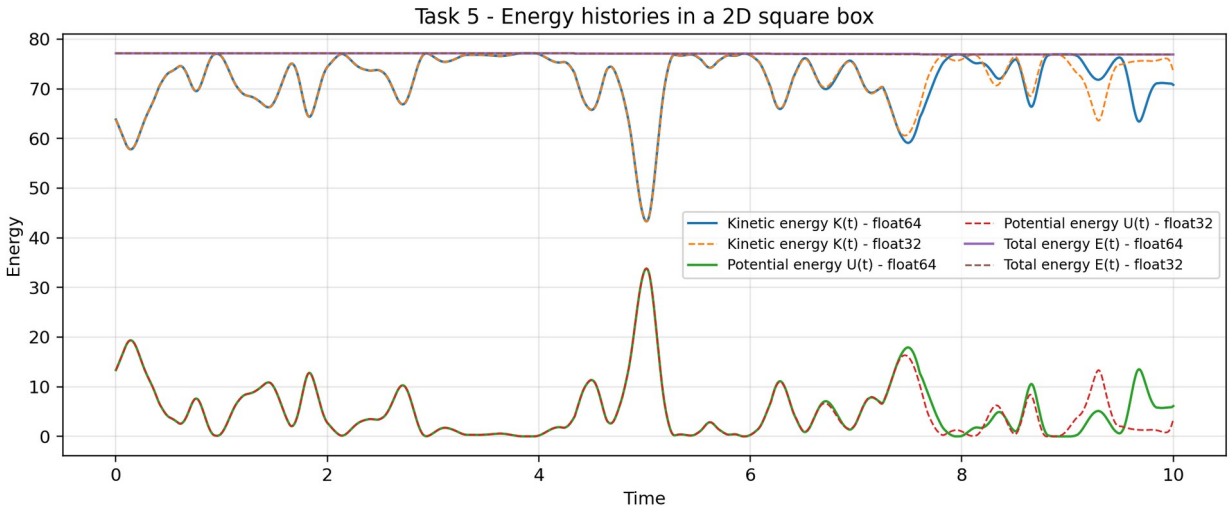


Figure 1. Kinetic, potential, and total mechanical energy histories for float32 and float64 in the 2D square box.

The maximum relative energy drift is 0.3029% for float32 and 0.3125% for float64. These values are close (about 0.0096 percentage points apart). The two energy histories follow similar trends. The step-like changes in total energy show that the measured drift is also affected by the time-stepping and boundary-update procedure; this experiment does not isolate those error sources completely, so the drift should not be attributed to floating-point precision alone.

4. Trajectory and runtime comparison

4.1 Difference in particle positions

The final aggregate relative position difference is epsilon_pos(T) = 0.498888, or 49.89%, with float64 used as the numerical reference. The measure compares the norm of the difference between the complete position arrays with the norm of the float64 position array. It is not a statement that every individual particle is wrong by this percentage.

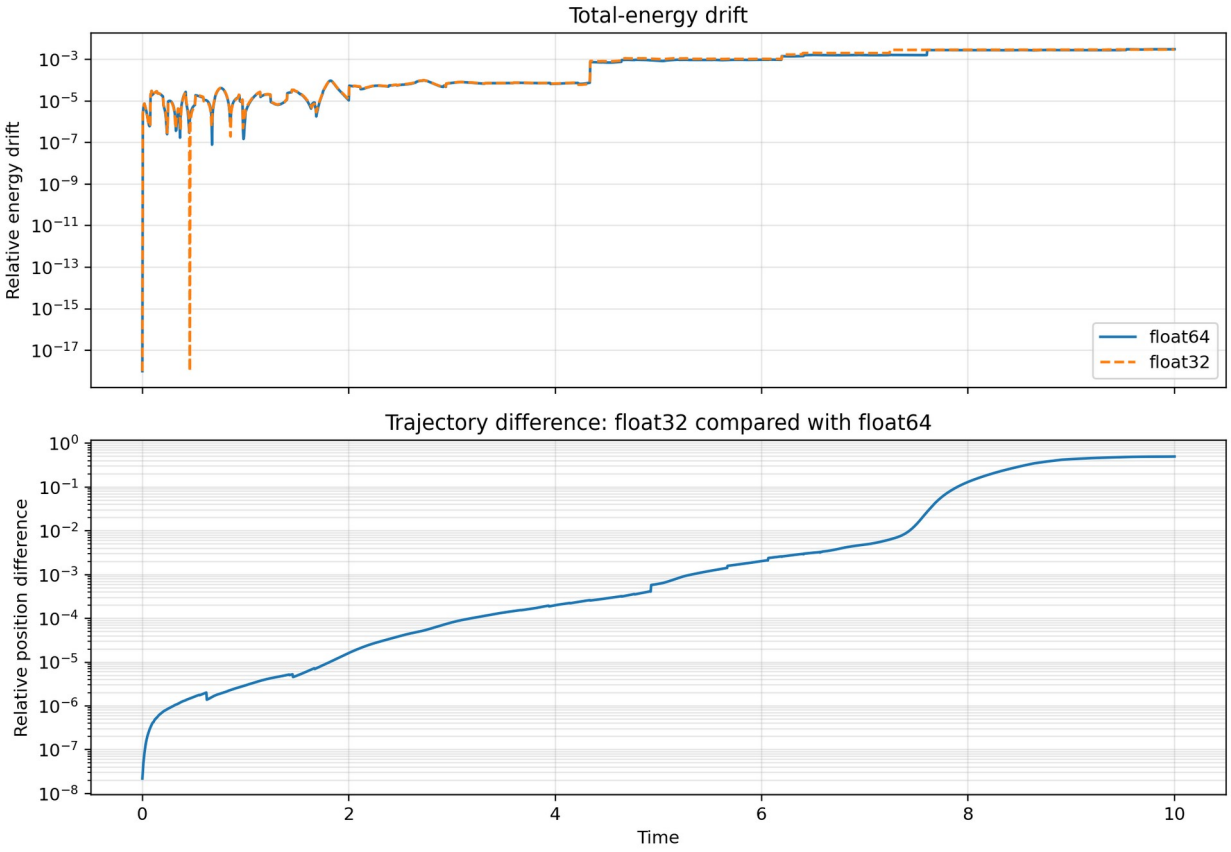


Figure 2. Relative energy drift and relative position difference over time. The position difference grows as the two numerical trajectories evolve.

The two runs begin from the same physical initial state, but casting and repeatedly rounding values in float32 produces slightly different forces and updates. Those small differences accumulate, and interacting-particle trajectories can become noticeably separated even when the total-energy histories remain close. float64 is a reference for this comparison, not an exact analytical solution.

4.2 Execution time

Repeat	float32 (s)	float64 (s)
1	0.206924	0.196522
2	0.199909	0.173453
3	0.210111	0.164923
4	0.547087	0.367270
5	0.283273	0.464669

Metric	float32	float64
Median runtime	0.210111 s	0.196522 s

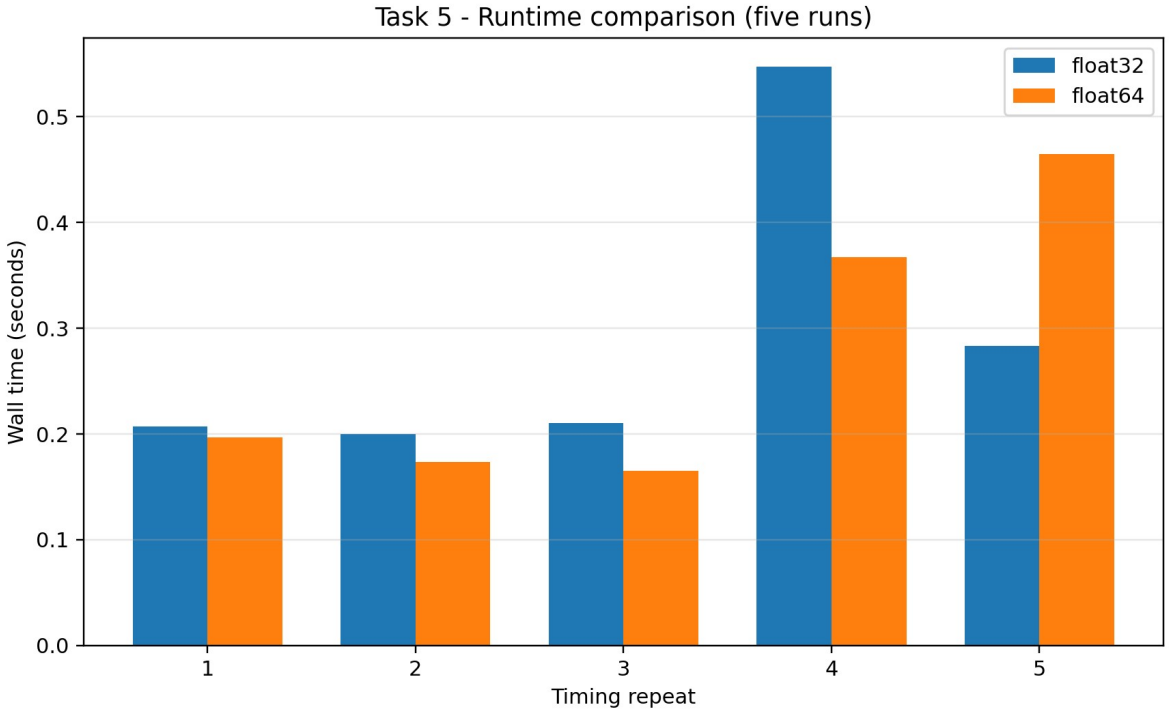


Figure 3. Five measured runtime samples for each precision. Median values are used in the comparison.

The median times are 0.210111 s (float32) and 0.196522 s (float64), a difference of about 6.47% relative to the slower result. At only 12 particles, the measured times are short and vary between repetitions. Therefore, this run does not demonstrate a meaningful runtime advantage for either precision. The runtime should be remeasured on the project computer before making a hardware-specific performance claim.

5. Discussion

5.1 What the measurements show

- Memory: float32 reduces tracked working-array memory from 4,272 bytes to 2,208 bytes, a reduction of 48.31%. Including the stored position history, the reduction is 49.98%.
- Runtime: the medians differ by only 0.013589 s. The difference is too small to support a general claim that one dtype is faster for this small simulation.
- Energy: the maximum relative energy drift is approximately 0.303% for float32 and 0.312% for float64. The measured energy behaviour is similar for the two formats in this run.
- Trajectory: the final aggregate relative position difference is 49.89%. Close energy values do not guarantee identical particle-by-particle trajectories.

5.2 Limitations and interpretation

- The experiment uses one seed and N=12 particles. The timing result is specific to the machine and software environment on which this run was executed.
- The working-memory figure counts the named NumPy arrays only. It excludes temporary arrays, Python overhead, plotting memory, and other process memory.
- The float64 trajectory is used as a comparison reference, not as a mathematically exact solution. A high trajectory difference can result from accumulated rounding and sensitivity of the interacting dynamics.
- The energy drift also depends on timestep, integration, and wall-reflection handling. It must not be interpreted as a pure floating-point error measurement.
- The uploaded simulator.py animation model and task3_4.py quantitative model are different variants. This report deliberately uses the latter for consistency with the Tasks 3-4 energy/integrator analysis.

6. Conclusion and engineering choice

For the current 2D, 12-particle experiment, float32 offers a clear memory benefit, but its runtime advantage is not established. Both precisions give similar total-energy drift, while their final particle positions differ substantially over the simulated interval. For the final scientific simulator, float64 is the recommended default when reliable trajectory comparison and numerical reproducibility are important. float32 can be considered for memory-constrained or larger runs only after repeating the accuracy tests at the chosen timestep and duration.

Answer to the Task 5 question

float32 is useful primarily because it halves the storage needed for floating-point values. In this experiment it appears adequate for the broad energy trend, but it does not reproduce the float64 particle trajectories over the full run. Its practical suitability therefore depends on whether the project prioritizes memory savings or close trajectory reproducibility.

Appendix A. How to reproduce on the project computer

- Place task5_precision_2d.py in the project folder alongside requirements.txt.
- Open the VS Code terminal in that folder and run: python -m pip install -r requirements.txt
- Run: python task5_precision_2d.py
- Open task5_2d_results/task5_precision_metrics.csv for the summary, task5_timing_samples.csv for all five timings, and task5_precision_timeseries.csv for step-by-step data.
- Use the newly generated results from the project computer in the final submission if they differ from the benchmark values in this report.

Benchmark environment: Python: 3.13.5 | NumPy: 2.3.5 | Matplotlib: 3.10.8 | Platform: Linux-6.18.57-x86_64-with-glibc2.41 | Processor: not reported by platform module